# C Sharp and Java Weighted Assessment

*Language features platform capabilities and decision weights*

Prepared • 25 October 2026

## Executive assessment

C# and .NET rank first on the assessed categories of the balanced enterprise model, at 86.0 versus 83.7 for Java and the JVM. The model covers 92 percent of assigned weight. The remaining 8 percent is reserved for benchmark performance and is not scored because current matched workload evidence is insufficient. These are qualitative model outputs, not objective measurements of universal superiority.

The expanded scope makes the result closer. Java receives credit for explicit modularity and a modest functional ecosystem breadth advantage. C# retains an expressiveness advantage and a modest core licensing clarity advantage. Framework capability, correctness, concurrency and deployment receive equal aggregate ratings with different underlying strengths. An integration-and-data weighting reverses the assessed ranking.

The published benchmark review includes ten historical CPU workloads and web-framework test methodology. It does not extrapolate .NET 9 versus Java 23 measurements to C# 15 versus Java 27. Missing benchmark evidence is neither a zero nor an assumed tie. The report preserves the full point-by-point language comparison and adds five platform domains, a scoring rubric, sensitivity cases and a completion protocol.

## Scope and independence

The user's opinion about C# is not evidence. Popularity, employment demand and IDE quality remain excluded. Ecosystem size and breadth, framework capability, licensing, deployment support and benchmark evidence are included. Download counts, job postings, language surveys, repository stars and IDE features contribute no score.

The language baseline is the documented C# 15 feature set and Java 27 as of 25 October 2026. C# 15 and selected safety changes are prerelease or preview. Production ecosystem and support judgments use documented supported product paths rather than pretending all C# 15 features are deployed everywhere. Exact total ecosystem size is unresolved; a disclosed functional breadth proxy receives a provisional rating. Facts, analyst ratings and missing evidence are distinguished throughout. [1–3, 60–85]

## Weighted ranking method

This model assumes a general enterprise application portfolio: backend services, transactional systems, integration and supporting user interfaces. It assumes no existing codebase, team experience, vendor agreement or cloud commitment. Those constraints can change the answer. The weights are analyst-selected priorities, not empirically discovered constants.

Language scoring uses the requested C# 15 feature set and Java 27. Production support and framework scoring use documented supported product paths, including .NET 10 LTS and Java 25 LTS where appropriate. This separation prevents crediting prerelease C# 15 features as already available across every production framework. Java 27 is a non-LTS release. Framework runtime compatibility must be checked independently. [1–3, 62–69]

| Rating | Meaning |
|---|---|

| Rating | Meaning |
|---|---|
| 0 | Unavailable or unusable for the stated criterion |
| 1 | Major blockers or exceptional implementation effort |
| 2 | Usable with substantial gaps or constraints |
| 3 | Capable with recurring workarounds or integration effort |
| 4 | Strong capability with manageable limitations |
| 5 | Exceptional fit with little material friction |

Half points interpolate between these anchors. Scores evaluate criterion fit, not the inherent worth of a language. Equal ratings indicate an equally strong aggregate assessment, not identical behavior. Ratings are qualitative judgments grounded in the report evidence. They are not measurements or probabilities. Weights total 100 percent; 53 percent concerns language semantics and architectural mechanisms and 47 percent concerns the added platform categories.

| Criterion | Weight | C# /5 | Java /5 | C# points | Java points |
|---|---|---|---|---|---|
| Language and abstraction | 18% | 4.5 | 3.5 | 16.2 | 12.6 |
| Correctness and contracts | 15% | 4.0 | 4.0 | 12.0 | 12.0 |
| Concurrency and resources | 10% | 4.5 | 4.5 | 9.0 | 9.0 |
| Modularity and evolution | 10% | 4.0 | 4.5 | 8.0 | 9.0 |
| Ecosystem size and breadth | 12% | 4.0 | 4.5 | 9.6 | 10.8 |
| Framework capabilities | 10% | 4.5 | 4.5 | 9.0 | 9.0 |
| Licensing clarity | 7% | 4.5 | 4.0 | 6.3 | 5.6 |
| Deployment and support | 10% | 4.5 | 4.5 | 9.0 | 9.0 |
| Benchmark performance | 8% | N/S | N/S | — | — |

N/S means not scored because comparable current-workload evidence is insufficient. It means neither zero nor a tie. The benchmark category keeps its 8 percent reserved weight; the report does not invent a performance rating to complete the table.

For each assessed category, weighted points = category weight × rating ÷ 5. Assessed index = 100 × total assessed points ÷ assessed weight. If every category is scored, the overall score is simply the sum of all weighted points. Values are calculated with decimal arithmetic and rounded only for display.

| Result | C# and .NET | Java and JVM |
|---|---|---|
| Assessed points | 79.1 of 92 | 77.0 of 92 |
| Assessed index | 86.0 of 100 | 83.7 of 100 |

| Result | C# and .NET | Java and JVM |
|---|---|---|
| Scored weight coverage | 92 percent | 92 percent |
| Rank on assessed categories | 1 | 2 |
| Full model bounds with benchmark rating from 0 to 5 | 79.1 to 87.1 | 77.0 to 85.0 |

C# leads on assessed categories by 2.1 weighted points, or about 2.3 normalized index points. These are model outputs, not statistically significant differences. The full-model bounds overlap. A Java benchmark rating more than 1.3125 points higher on the 0-to-5 scale would reverse the baseline result, assuming other ratings stay fixed. The bounds are mathematical possibilities for the missing category, not confidence intervals or forecasts.

## Reasons for the ratings

**Language and abstraction.** C# 4.5; Java 3.5. C# directly supports more of the representations and API forms in the feature comparison. Java remains capable, but erased generics, absent user-defined value types and operators, and lack of native async and generator syntax cause recurring supporting implementations. C# stops short of 5 because feature interactions, partial expression-tree coverage and union representation limits remain. C# 15-specific credit is provisional.

**Correctness and contracts.** Both 4.0. C# provides native null-state assistance and initialization mechanisms. Java provides useful final-binding, checked-exception and exhaustive-switch restrictions. Neither establishes complete domain validity or general concurrency safety. A tie deliberately avoids counting every convenience construct as an enforced guarantee.

**Concurrency and resources.** Both 4.5. C# has direct async, iterator and asynchronous-disposal support. Java combines virtual threads, synchronous cleanup with suppressed failures and preview task scopes. No category bonus is inferred from claimed throughput. The Java preview task-scope contribution must be reconsidered if preview use is prohibited.

**Modularity and evolution.** C# 4.0; Java 4.5. Java module contracts and sealed interfaces provide explicit architectural boundaries. C# supplies assembly access and partial source composition, but lacks an equivalent package-export and reflective-opening declaration. This is a design-capability judgment, not proof of lower upgrade cost.

**Ecosystem size and breadth.** C# 4.0; Java 4.5. The sampled inventory supports a modest Java depth advantage for enterprise integration and distributed data processing, while .NET has strong application, UI and game libraries. Exact total ecosystem size is unresolved. The half-point difference is a low-to-medium-confidence functional proxy and should be rescored against an organization’s actual dependency list. [60–61, 70–76]

**Framework capabilities.** Both 4.5. ASP.NET Core, EF Core, Blazor and .NET UI choices provide broad application workflows. Spring and Quarkus provide broad enterprise and service workflows. Neither covers every application domain with one uniform runtime and lifecycle. Framework fitness can change substantially by workload. [70–76]

**Licensing clarity.** C# 4.5; Java 4.0. The modest .NET advantage concerns the straightforward MIT core distribution model. OpenJDK GPL with Classpath Exception permits a viable no-fee commercial application path; Oracle licensing is optional and version-dependent. Java is not penalized as though Oracle subscriptions were mandatory. Selecting one approved OpenJDK distribution can justify raising Java to 4.5. Third-party products on either side have separate terms. [77–81]

**Deployment and support.** Both 4.5. Both have mainstream cross-platform, container, runtime-bundled and native deployment paths. .NET has integrated publishing options; Java has modular runtime images, multiple distribution choices and longer optional commercial LTS windows. AOT restrictions and target-specific compatibility remain on both sides. [62–69]

**Benchmarks.** Not scored. Historical published results are useful evidence but do not establish current C# 15 versus Java 27 performance for an unspecified enterprise workload. No new application benchmark was run. The completion protocol below makes this category usable once targets and measurements exist. [82–85]

## Sensitivity to priorities and uncertain ratings

The following profiles retain the same ratings and change only the weights. Vector order is language, correctness, concurrency, modularity, ecosystem, frameworks, licensing, deployment, benchmarks. Each vector totals 100. Benchmark evidence remains unscored, so index coverage is 92 percent for the baseline and 90 percent for the alternatives.

| Profile | Weight vector percent | C# index | Java index | Assessed leader |
|---|---|---|---|---|
| Balanced enterprise | 18/15/10/10/12/10/7/10/8 | 86.0 | 83.7 | C# |
| Language and correctness | 30/20/10/10/5/5/5/5/10 | 86.1 | 80.6 | C# |
| Integration and data | 10/15/10/15/20/10/5/5/10 | 84.4 | 85.6 | Java |
| Deployment emphasis | 10/10/10/10/10/10/10/20/10 | 86.7 | 85.6 | C# |

The integration-and-data profile reverses the ranking. Under the balanced profile, lowering C# language capability by half a point reduces its lead from 2.1 to 0.3 raw points. Raising Java licensing to 4.5 reduces the baseline lead to 1.4 points. Reducing C# language by half a point and removing the licensing distinction together puts Java ahead by 0.4 points. Those changes are plausible judgments, so the narrow lead is not robust.

Avoid double counting. Ecosystem rates breadth and alternatives, frameworks rate workflow capability, deployment rates packaging and support, benchmarks rate measured outcomes, and licensing rates terms and predictability. When one feature appears in multiple original domains, score its expressiveness only under language and its enforceable guarantee only under correctness. A feature being mentioned twice does not earn two independent bonuses.

Before using totals, apply non-compensatory gates: required platform support, acceptable licenses for the actual bill of materials, supported production release status, mandated functionality and hard service-level limits. A failing gate is not offset by a good score elsewhere. A production requirement for C# 15-only features cannot be satisfied today merely by assigning .NET 10 a high deployment score.

## Assessment by domain

| Domain | Assessment | Reason |
|---|---|---|
| Core types and arithmetic | C# | More direct representation and arithmetic controls |
| Nullability and initialization | C# with qualifications | Built-in null-state analysis; Java has stronger final-binding mechanisms |
| Generics | C# overall | Reification and generic operators; Java leads in use-site variance and intersection casts |

| Domain | Assessment | Reason |
|---|---|---|
| Object-oriented APIs | C# | More direct API concepts and composition mechanisms |
| Data and closed domains | C# overall | Unions and representation choices; Java retains rich enums and sealed interfaces |
| Patterns and control flow | Split | C# breadth; Java stricter default exhaustiveness |
| Functions and code as data | C# overall | Local functions and compiler-generated expression trees |
| Iteration and concurrency | Split across layers | C# resumable syntax; Java virtual threads and preview task scopes |
| Errors and cleanup | Split | Different contracts, filtering, asynchronous cleanup and failure preservation |
| Memory and native access | C# for direct language control | Both platforms have capable native APIs |
| Everyday ergonomics | C# overall | More built-in construction and call-site syntax |
| Modules and source composition | Split | Java module contracts; C# partial declarations and dynamic syntax |

# 1 Core types and numeric semantics

Representation choices affect copying, identity, arithmetic policy, and generic storage.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| User-defined values | Structs and record structs alongside reference classes. | Primitives are values; ordinary classes and records are references. | C# provides more control. |
| Integral types | Signed, unsigned, and native-sized integers. | Primarily signed integers; unsigned char and additional unsigned APIs. | C# has broader native support. |
| Decimal arithmetic | Built-in fixed-precision decimal, literals and operators. | Arbitrary-precision BigDecimal API. | C# syntax; Java precision range. |
| Overflow | checked and unchecked control integral overflow policy. | Ordinary arithmetic wraps; exact arithmetic methods can detect overflow. | C# language integration. |
| Parameters | By value by default; explicit ref, out and readonly-reference mechanisms. | Always passes values, including copies of object references. | C# flexibility; Java uniformity. |

Neither language detects all integral overflow by default. BigDecimal and decimal are different numeric models. C# value types do not imply stack allocation or an automatic performance win. Both languages can mutate an object through a passed reference; C# additionally supports passing the caller’s variable by reference.

**Sources [4–8].**

## 2 Nullability and initialization

The relevant distinction is the guarantee established by each construct, rather than its name.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Reference nullability | Nullable annotations and compiler null-state analysis. | Reference types permit null; annotations alone impose no core nullness checking. | C# native analysis advantage. |
| Null handling | Null-conditional access and coalescing operators. | Explicit checks, conditional expressions and Optional APIs. | C# concise expression. |
| Optional values | T? for built-in and user-defined value types. | Nullable wrappers and optional-value APIs. | C# consistent language syntax. |
| Construction obligations | required members and init-only accessors. | Constructor parameters and definite assignment of blank final fields. | Different construction models. |
| Final bindings | readonly in selected contexts; no general final modifier for ordinary locals. | final locals, parameters and fields. | Java binding immutability. |

Nullable reference types generate diagnostics and do not change runtime types. required establishes ordinary construction-site obligations, not business validity. readonly and final references do not freeze referenced objects. Default-initialized C# structs can bypass constructor validation. Java's blank-final rules enforce assignment and prevent reassignment.

**Sources [9–13].**

## 3 Generics and generic algorithms

Generic capability includes runtime identity, representation, constraints, variance and available operations.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Runtime identity | Constructed generic arguments remain available at runtime. | Parameterized types generally undergo erasure; declarations can retain metadata. | C# fewer runtime restrictions. |
| Value arguments | List<int> stores elements without per-element boxing. | List<Integer>; primitives are not generic arguments. | C# representation advantage. |
| Generic construction | new T() with a suitable constraint; arrays of T. | No direct new T() or new T[]; factories and other mechanisms are used. | C# implementation advantage. |
| Variance | Declaration-site in and out on interfaces and delegates for reference arguments. | Use-site extends and super wildcards. | Java flexibility for consuming generic classes. |
| Static contracts | Static abstract interface members support operator and static-method requirements. | Bounds express class and interface relationships; strategies supply operations. | C# generic algorithm advantage. |

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Intersection types | Multiple interface constraints, without general intersection-cast syntax. | Intersection bounds and casts. | Java advantage in this dimension. |

C# leads overall through reification, value-type support and static polymorphism. Java's wildcard flexibility is substantive. Erasure does not eliminate Java's compile-time generic type checking or every piece of generic declaration metadata. Mutable generic list types are not freely covariant in either language.

**Sources [4, 14–17].**

## 4 Object oriented APIs

The languages share a class and interface foundation, but C# gives more API concepts dedicated syntax.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Inheritance | Single class inheritance, multiple interfaces and default interface members. | Broadly comparable mechanisms. | Broad parity. |
| Overriding | Ordinary class methods are non-virtual by default; virtual and abstract expose extension points. | Overridable instance methods are the usual default; final prevents overriding. | Different extension policies. |
| Interface-specific behavior | Explicit interface implementations can separate identical member signatures. | Shared implementations or adapters normally provide alternatives. | C# advantage. |
| Properties and indexing | Dedicated properties and indexers, including automatic accessors. | Methods and conventions. | C# direct expression. |
| Extensions | Extension methods, properties, operators and C# 15 indexers. | Static helpers, adapters or changes to declarations. | C# API composition advantage. |
| Events | Dedicated subscription members with publisher-controlled notification. | Listener and observer interfaces and methods. | C# native support. |
| Operators | User-defined operators and conversions. | No general user-defined operator overloading. | C# expression; Java fixed semantics. |

C# more directly expresses API intent. Its overriding default makes extension permission explicit, while Java makes extensibility easier. Extensions do not alter the original type or grant private access. Operator overloading improves domain notation when meanings remain predictable; surprising conversions and operators add reading cost.

**Sources [18–23].**

# 5 Data and closed domains

C# 15 substantially improves composition of alternative outcomes, while Java retains useful restrictions.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Records | Record classes and structs; record-class inheritance and value-oriented behavior. | Final reference records with generated accessors, equality and formatting. | C# options; Java narrower abstraction. |
| Immutability | Readonly structs and fields, init accessors; records may be mutable. | Final components and fields. | Neither implies deep immutability. |
| Small aggregates and copies | Tuples, deconstruction and with expressions. | Named records, constructors and methods. | C# concise composition. |
| Unions | Named closed sets of existing case types without shared inheritance. | Sealed hierarchies and wrappers model alternatives. | C# composition advantage. |
| Closed inheritance | closed classes restrict direct subtypes to their assembly. | Sealed classes and interfaces specify permitted subtypes. | Split; Java offers sealed interfaces. |
| Enums | Numeric types support codes and flags; unnamed values are possible. | Constants can carry state, implement interfaces and provide behavior. | Java rich-domain advantage. |
| Equality | Records supply value-oriented ==; other types can overload it. | Reference == tests identity; records provide value-oriented equals. | Natural notation versus uniform identity distinction. |

C# with expressions are shallow copies, and neither language's records automatically freeze reachable objects. Generated C# unions store object? contents, box value-type cases and can contain null when default-initialized. closed constrains direct descendants; further inheritance can remain open. Java permits deliberate reopening through non-sealed subtypes. C# leads in modeling options, with important Java exceptions.

**Sources [24–30].**

# 6 Patterns and control flow

Pattern breadth and enforcement should be judged independently.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Pattern vocabulary | Type, property, positional, relational, logical and list patterns. | Type and record patterns, guards, plus preview primitive patterns. | C# breadth advantage. |
| Deconstruction | Positional patterns can use a suitable Deconstruct method. | Record patterns decompose records, including nested records. | C# general adaptation; both support nesting. |
| Closed-set coverage | Recognizes exhaustive unions and closed-hierarchy switch expressions. | Coverage of enums, sealed hierarchies and record-pattern combinations. | Both have substantive analysis. |
| Enforcement | Incomplete switch expressions generally warn; ordinary switches need not be exhaustive. | Incomplete switch expressions and enhanced switch statements are errors. | Java default enforcement advantage. |

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Nested-loop exits | C# 15 adds labeled break and continue. | Longstanding labeled break and continue. | Broad parity for loop control. |

C# has a richer pattern vocabulary. Java has stricter default enforcement for enhanced switches. C# projects can elevate relevant warnings to errors. Older non-enhanced Java switch statements need not be exhaustive. A catch-all branch can also conceal newly added cases, so exhaustive modeling depends on how switches are written.

**Sources [1–3, 31–32].**

## 7 Functions queries and code as data

Both support passing behavior; C# provides more mechanisms around that foundation.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Anonymous functions | Lambdas and method groups target delegates. | Lambdas and method references target functional interfaces. | Broad parity in passing behavior. |
| Capture | Mutable locals can be captured; static lambdas prohibit capture. | Captured locals must be final or effectively final. | C# flexibility; Java stricter bindings. |
| Local functions | Named local helpers, including generic, async and iterator functions. | Lambdas and local classes provide alternatives. | C# direct support. |
| Queries | Language query syntax translates to query-pattern method calls. | Stream pipelines use APIs and lambdas. | C# language integration. |
| Expression trees | Supported expression lambdas can be captured as inspectable code structures. | No comparable general compiler conversion of ordinary lambdas into expression trees. | C# interpretable API advantage. |

Java's capture restriction does not make captured objects immutable or thread-safe. C# static lambdas explicitly prevent capture. Expression trees are more consequential than query notation alone because APIs can inspect supplied expressions, but compiler capture supports a subset of C# syntax. Query syntax does not guarantee arbitrary provider translation or efficient execution.

**Sources [33–36].**

## 8 Iteration asynchronous execution and concurrency

Language-supported suspension and runtime-supported lightweight threads are different mechanisms.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Generators | yield return generates resumable iterator state. | Iterator and stream producers use APIs and implementations. | C# compiler support. |
| Async methods | async and await support sequential-looking asynchronous workflows. | Futures and completion stages provide API composition. | C# language integration. |

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Async sequences | Async iterators and await foreach. | API-based approaches without equivalent iterator syntax. | C# language advantage. |
| Blocking-style concurrency | Often distinguishes synchronous and asynchronous operations. | Virtual threads support many blocking-style I/O tasks over fewer platform threads. | Java runtime strength. |
| Task lifetimes | Task composition alone does not impose lexical ownership of child tasks. | Preview StructuredTaskScope groups tasks and bounds their lifetimes. | Java preview API advantage. |
| Cancellation | Cooperative cancellation tokens. | Interruption and related task APIs. | Both depend on cooperation. |
| Mutual exclusion | Language lock and runtime locking mechanisms. | Language synchronized and monitors. | Broad parity for basic exclusion. |

C# provides stronger syntax and compiler support for resumable workflows. await suspends an async method rather than blocking its evaluating thread. Java virtual threads can preserve ordinary blocking call chains, especially for I/O-heavy concurrency. Structured concurrency adds task-lifetime organization but remains preview. Neither approach establishes a general CPU-speed advantage or proves programs free of races and deadlocks.

**Sources [3, 37–42].**

## 9 Error contracts and resource management

Different mechanisms provide different guarantees; this domain has no unconditional winner.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Checked failures | Thrown exception types are not enforced in signatures. | Checked exceptions require catch-or-declare handling. | Java enforceable contract; composition cost. |
| Returned outcomes | Unions directly compose result alternatives. | Sealed result hierarchies model alternatives. | C# syntax and composition. |
| Catch policies | Boolean when filters. | Multi-catch; further tests normally run in handlers. | Different useful mechanisms. |
| Scoped cleanup | using statements and declarations. | Try-with-resources. | Broad parity. |
| Async cleanup | Dedicated await using. | AutoCloseable supplies a synchronous closing contract. | C# explicit async support. |
| Competing failures | Disposal failures can replace an already propagating failure without explicit handling. | Cleanup failures are automatically retained as suppressed exceptions. | Java preservation advantage. |

Returning a C# union does not constrain all exceptions a method can throw. Java checked contracts likewise do not enumerate unchecked exceptions. C# exception filters run before stack unwinding, unlike conditions inside a handler. Java suppressed exceptions address a separate correctness concern during cleanup.

**Sources [27, 32, 43–45].**

# 10 Memory control and native access

C# provides more direct language control; modern Java also has substantial native-access APIs.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Managed memory | Garbage collection with user-defined value representations. | Garbage collection with primitive and object representations. | No general speed verdict. |
| Stack allocation | Explicit stackalloc, including safe use through spans. | No equivalent general source-level stack-allocation expression. | C# allocation control. |
| Lifetimes | ref struct restrictions prevent certain heap escapes. | Foreign-memory arenas and segments control access and lifetime through APIs. | Compiler versus API and runtime controls. |
| Pointers | Opt-in pointers and function pointers; preview revised safety obligations. | Foreign-memory APIs rather than ordinary pointer syntax. | C# low-level expression. |
| Native functions | P/Invoke and generated bindings. | Foreign Function and Memory API, alongside JNI. | Both capable; workload-dependent suitability. |

Value semantics, spans and stack allocation offer options rather than universal allocation or speed guarantees. C# lifetime restrictions do not amount to a general ownership proof. Java should receive credit for current foreign-memory and function APIs, rather than being evaluated only through JNI. Native calls introduce safety obligations on both platforms.

**Sources [1, 46–49].**

# 11 Everyday syntax and construction

Convenience features reduce ceremony, with importance determined by actual usage.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Inference | var infers a static local type. | var infers a static local type. | Broad parity. |
| Call sites | Named and optional arguments. | Overloads, configuration objects and builders. | C# direct expression. |
| Strings | Interpolation, raw strings and handler-based customization. | Text blocks and formatting APIs. | C# language integration. |
| Collections | Target-typed expressions, spreads and C# 15 constructor or builder arguments. | Array initializers and collection factory APIs. | C# general construction syntax. |
| Constructors | Primary constructors on ordinary classes and structs, plus records. | Concise record construction and flexible constructor bodies. | Different conveniences; broader C# shorthand. |
| Small programs | Top-level statements. | Compact source files and instance main methods. | Both reduce ceremony. |

C# primary-constructor parameters do not automatically become public properties on ordinary classes. Java now permits suitable constructor statements before explicit superclass invocation and supports low-ceremony small programs. These current capabilities prevent outdated comparisons based on historical boilerplate. Convenience is meaningful but not equivalent to greater computational power.

**Sources [1–2, 50–54].**

## 12 Modules metadata and source composition

Java modules and C# source composition solve distinct architectural needs.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Internal boundaries | Assembly-scoped internal, combined access scopes and friend assemblies; namespaces organize names. | Package access and named modules provide distinct boundaries. | Different boundary models. |
| Module contracts | No directly equivalent language declaration for package exports, reflective opening and dependencies. | requires, exports, opens and service relationships. | Java declarative modularity advantage. |
| Metadata | Attributes and reflection. | Annotations and reflection, including retention policies. | Broad parity in purpose. |
| Dynamic binding | dynamic defers selected binding checks to runtime. | Reflection and method-handle APIs, without equivalent ordinary dynamic-member syntax. | C# direct expression with fewer static checks. |
| Source composition | Partial types and members combine generated and handwritten declarations. | Generated additional or companion types. | C# language composition advantage. |
| Generation | Compiler-platform source generators. | Annotation-processing APIs. | Both capable compiler and API facilities. |

Java provides a more explicit language vocabulary for module dependencies, exports and reflective access. C# assemblies and friend access are useful but not interchangeable with that model. C# offers stronger direct source composition. Both support metadata-driven generation, so attributing that entire capability to either language would be inaccurate.

**Sources [18, 55–59].**

## 13 Ecosystem size and functional breadth

Size is assessed through qualified capability coverage and alternatives rather than downloads or developer headcount.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Registry evidence | NuGet distinguishes package identity from package versions. | Maven Central contains JVM artifacts, including languages beyond Java. | Raw counts are not directly comparable. |

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Enterprise integration | Application APIs and libraries support common enterprise dependencies. | Spring Integration and Apache Camel supply extensive documented integration mechanisms. | Sample supports a Java depth advantage. |
| Distributed data | C# applications can use services and available clients. | Spark offers native Java APIs; Flink provides distributed stream processing. | Java has a stronger documented native path in this sample. |
| UI and games | .NET UI frameworks and C# game-engine integration. | Java UI and application stacks remain viable. | C# strengths offset some backend breadth differences. |
| Score confidence | 4.0 of 5; low to medium confidence. | 4.5 of 5; low to medium confidence. | A functional proxy rather than a full census. |

There is no defensible claim here that Java has a specific multiple of .NET libraries. Counts must deduplicate versions, artifacts from one project, forks and language-irrelevant packages, then consider maintenance and target compatibility. Download volumes would reintroduce the excluded popularity criterion. This inventory is representative, not an exhaustive audit. Client access to a service is not automatically equivalent to native framework support, but it is also not absence of capability.

**Sources [60–61, 70–76].**

## 14 Framework capabilities

Framework capability is scored separately from framework quantity.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Web APIs | ASP.NET Core routing, server APIs and web application mechanisms. | Spring and Quarkus service frameworks. | Both strong for enterprise services. |
| Persistence | EF Core provides object mapping and database-provider integration. | Spring Data and persistence integrations cover multiple data stores. | Strong alternatives; provider behavior matters. |
| Browser UI | Blazor supports C# web UI workflows. | Java stacks can expose APIs and support server-driven or separate browser UI. | C# integrated C# browser path. |
| Native UI | .NET MAUI targets Android, iOS, Windows and macOS through Mac Catalyst; Avalonia supplies further choices. | Java desktop options and mobile environments have separate runtime constraints. | Framework targets are not identical to runtime targets. |
| Data and integration | Broad service clients and application integrations. | Spring Batch, Spring Integration, Camel, Spark and Flink cover substantial integration and data workflows. | Java stronger in this sampled specialization. |
| Aggregate rating | 4.5 of 5. | 4.5 of 5. | Different strengths; no universal framework leader. |

A framework's existence does not prove compatibility with every C# 15 or Java 27 feature. For example, the retrieved Spark documentation enumerates Java 17, 21 and 25 rather than Java 27. Game engines and mobile runtimes also have their own language and runtime support. Native Linux desktop UI support should not be inferred from .NET MAUI's target list; Avalonia is a separate choice.

**Sources [70–76].**

## 15 Licensing and commercial terms

The baseline distinguishes language specifications, runtime distributions, frameworks, optional support and proprietary products.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Core runtime | .NET runtime source is MIT-licensed. | OpenJDK distributions use GPLv2 with Classpath Exception. | Both offer no-fee commercial application paths. |
| Application obligations | Preserve relevant license and copyright notices; third-party components retain terms. | Classpath Exception addresses linking with independent modules; redistribution obligations still matter. | Java GPL should not be treated as automatic application copyleft. |
| Oracle distribution | Not applicable to the .NET runtime choice. | Oracle JDK 25 and later currently have NFTC no-fee terms, subject to conditions and update policies. | Oracle JDK is not the only Java distribution. |
| Updates and support | Public lifecycle and separate support options. | Distribution and version affect update licensing and commercial support. | A Java distribution policy can simplify administration. |
| Framework licensing | ASP.NET Core core license is MIT; other products vary. | Spring Boot license is Apache 2.0; other products vary. | No blanket claim that all ecosystem software is free. |
| Rating | 4.5 of 5. | 4.0 of 5; 4.5 is reasonable with a fixed approved OpenJDK policy. | Small clarity preference, not a mandatory cost penalty. |

This is a comparison of published terms and policy clarity, not legal advice about a particular deployment. The actual license and bill of materials control. Oracle's FAQ says Java 27 NFTC covers its planned six-month support life and describes a future license transition for Oracle JDK 25 updates after its no-fee window. Those Oracle-specific terms must not be generalized to every OpenJDK distribution. Commercial engine, UI, support and cloud products can add separate costs on either side.

**Sources [77–81, 67].**

## 16 Deployment support and lifecycle

Support is evaluated against actual runtime versions, distributions, operating systems and packaging modes.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Cross-platform runtime | .NET runs across mainstream desktop and server targets; exact release matrices apply. | OpenJDK distribution matrices cover mainstream and additional target combinations. | Vendor and version must be named. |

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Runtime-bundled deployment | Self-contained and single-file publishing options. | jlink runtime images and jpackage application packaging. | Both can avoid a separately installed system runtime. |
| Containers | Documented .NET deployment workflows. | Documented JDK and framework container workflows. | Both capable; image size is a measured concern. |
| Native compilation | .NET Native AOT has dynamic-loading, generation and trimming constraints. | GraalVM Native Image uses reachability analysis and metadata for dynamic features. | Both require compatibility work. |
| Support horizon | .NET 10 LTS public support ends November 2028 under the current policy. | Oracle Java 25 commercial roadmap extends to 2030 Premier and 2033 Extended support. | Different free and paid support arrangements. |
| Rating | 4.5 of 5. | 4.5 of 5. | Balanced fit; specific targets can change the result. |

Java 27 is non-LTS and Oracle's roadmap lists support through March 2027. That is not the same lifecycle as Java 25 LTS. Likewise, C# 15's prerelease status cannot be hidden behind .NET 10 support. Bundled runtimes transfer patching responsibility into application delivery. A portable managed artifact, a native binary and a platform-specific installer have different portability rules; jpackage packages are built on their target platform.

**Sources [62–69].**

## 17 Benchmarks and performance evidence

Published measurements are included, but no current enterprise workload score is invented.

| Feature | C# 15 | Java 27 | Assessment |
|---|---|---|---|
| Public microbenchmarks | Retrieved Benchmarks Game page uses .NET SDK 9.0.100, runtime 9.0.0 and JIT execution. | The paired page uses Java 23 HotSpot. | Historical evidence, not C# 15 versus Java 27. |
| Web framework tests | ASP.NET implementations can be benchmarked under shared specifications. | Java framework implementations can use the same specifications. | Framework and implementation comparison, not pure language measurement. |
| Measurement dimensions | Throughput, tail latency, cold start, resident memory and CPU efficiency. | Same dimensions, resources and correctness requirements. | Do not collapse different metrics without declared priorities. |
| Evidence admission | Exact versions, source, build flags, hardware, workload and repeated trials. | Same required evidence. | Optimized and ordinary implementations must be distinguished. |
| Reserved model weight | 8 percent; not scored. | 8 percent; not scored. | Unknown is neither zero nor a tie. |

The public examples below are sufficient to show implementation-sensitive results and insufficient to establish a current workload winner. No new application benchmark was run. TechEmpower Round 23 is historical and its tests cover different amounts of application work; Fortunes is a richer database-and-template exercise than plaintext. No unverified throughput number or modern-runtime extrapolation is used in the ranking.

**Sources [82–85].**

## Published benchmark observations

The table shows the lowest displayed valid elapsed time per language for every workload on the retrieved page. Flagged implementations are included; the final three workloads are marked contentious. These unmatched programs receive no aggregate score. [82]

| Workload | C# elapsed seconds | Java elapsed seconds |
|---|---|---|
| fannkuch-redux | 2.35 | 10.48 |
| n-body | 3.20 | 6.92 |
| spectral-norm | 0.96 | 1.61 |
| mandelbrot | 3.23 | 4.16 |
| fasta | 1.23 | 1.20 |
| k-nucleotide | 3.26 | 4.94 |
| reverse-complement | 0.68 | 3.20 |
| binary-trees | 6.40 | 2.62 |
| pidigits | 0.84 | 0.84 |
| regex-redux | 1.43 | 1.60 |

The measurement guide describes elapsed minima from repeated runs on an older quad-core machine. These examples do not establish current production latency, throughput per dollar or a universal language speed ratio. [83]

## Protocol for completing the benchmark score

Agree workload and hard service-level objectives before testing. Hold hardware, CPU and memory limits, operating system, database, payloads, correctness, external services, logging and optimization policy constant. Record exact runtime and framework versions, source revisions, compiler flags and JIT or AOT mode. Measure cold start separately from warmed operation, alternate run order, retain repetitions and variability, and preserve raw measurements. Compare equivalent database and ORM behavior rather than raw-driver code on one side and full-stack ORM code on the other.

| Metric | Share of benchmark category | Direction |
|---|---|---|
| Sustained API throughput | 20 percent | Higher is better at the required error rate and latency |
| Tail latency at fixed offered load | 30 percent | Lower p99 is better |
| Cold start to readiness | 15 percent | Lower is better |
| Steady resident memory | 15 percent | Lower is better under equal resource limits |

| Metric | Share of benchmark category | Direction |
|---|---|---|
| CPU task completion time | 20 percent | Lower is better for identical results |

For throughput, achievement ratio R = observed throughput divided by required minimum. For latency, startup, memory and CPU duration, R = permitted maximum divided by observed value. Score each metric using the bands below, then take the metric-weighted average to obtain the 0-to-5 benchmark rating. This measures headroom against the same declared requirements. A hard service-level failure still triggers the gate; a total cannot compensate for it.

| Achievement ratio R | Metric rating |
|---|---|
| R below 0.50 | 0 |
| 0.50 up to but excluding 0.75 | 1 |
| 0.75 up to but excluding 1.00 | 2 |
| 1.00 up to but excluding 1.25 | 3 |
| 1.25 up to but excluding 1.67 | 4 |
| R of 1.67 or greater | 5 |

Budgets, offered load and measurement confidence are currently unspecified, so this protocol produces no numeric rating yet. Mark an untested submetric N/S and report its missing weight rather than silently redistributing it. Small differences within measurement noise should receive the same rating. Once all benchmark metrics are scored, contribution = 8 × benchmark rating ÷ 5, and the full 100-point result can be calculated.

## Decision interpretation

The weighted model favors C# under the stated balanced-enterprise priorities, but the 2.1-point assessed lead is sensitive to reasonable ratings and weight changes. Java leads when integration, data ecosystem and modularity receive greater importance. No full-model winner has been established because benchmark evidence remains unscored. The correct conclusion is a conditional ranking with transparent assumptions.

Use the original twelve domains for detailed language design differences and the five added domains for platform judgments. Apply hard requirements before totals, validate actual dependencies and distribution terms, and replace the provisional ecosystem and performance inputs with evidence from the intended workload. Maintenance cost, defect rates and application performance cannot be inferred from feature counts or this qualitative index alone.

## Source references

Primary sources from Microsoft, Oracle and the official Java developer documentation. URLs correspond to sources reviewed during the evaluation. Version and preview status are anchored to 25 October 2026. Number ranges refer to every source in the stated range. The linked documents may evolve after this report date.

[1] C# 15 feature set

https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-15

[2] Java language changes

https://docs.oracle.com/en/java/javase/27/language/java-language-changes-summary.html

[3] Java 27 changes and previews

https://docs.oracle.com/en/java/javase/27/migrate/significant-changes-jdk-27-release.html

[4] Core type models

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/struct

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-4.html

[5] Integral types

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types

[6] Decimal models

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/math/BigDecimal.html

[7] Overflow behavior

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/checked-and-unchecked

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/lang/Math.html

[8] Method arguments

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/method-parameters

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-15.html

[9] Nullable references

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/null-safety/nullable-reference-types

[10] Optional values

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/nullable-value-types

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/Optional.html

[11] Initialization members

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/required

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/init

[12] Definite assignment

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-16.html

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/variables

[13] Readonly bindings

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/readonly

[14] C# generic types

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/generics

[15] Java generic restrictions

https://dev.java/learn/language/fp/generics/restrictions/

[16] Variance

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/in-generic-modifier

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/out-generic-modifier

https://dev.java/learn/language/fp/generics/wildcards/

[17] Generic constraints and static members

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/where-generic-type-constraint

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/interface-implementation/static-virtual-interface-members

[18] Java interfaces and annotations

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-9.html

[19] Overriding policies

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/virtual

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-8.html

[20] Explicit interface implementation

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/interfaces/explicit-interface-implementation

[21] Properties and indexers

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/properties

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/indexers/

[22] Extension members

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/extension-methods

[23] Events and operators

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/event

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/operator-overloading

[24] C# records

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/record

[25] Java records

https://docs.oracle.com/en/java/javase/27/language/records.html

[26] Tuples

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/value-tuples

[27] Union types

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/union

[28] Closed hierarchies

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/closed

https://docs.oracle.com/en/java/javase/27/language/sealed-classes-interfaces.html

[29] Enum models

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/enum

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-8.html#jls-8.9

[30] Equality

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/equality-operators

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-15.html#jls-15.21

[31] Pattern matching

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/patterns

https://docs.oracle.com/en/java/javase/27/language/pattern-matching-switch.html

[32] Java statements and cleanup

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-14.html

[33] Lambdas and capture

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/lambda-expressions

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-15.html#jls-15.27

[34] Local functions

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/building-software/local-functions

[35] Query expressions and streams

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/expressions

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/stream/package-summary.html

[36] Expression trees

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/expression-trees/

[37] Iteration and suspension

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/yield

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/await

[38] Futures

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/concurrent/CompletableFuture.html

[39] Virtual threads

https://docs.oracle.com/en/java/javase/27/core/virtual-threads.html

[40] Structured concurrency preview

https://docs.oracle.com/en/java/javase/27/core/structured-concurrency.html

[41] Cancellation

https://learn.microsoft.com/en-us/dotnet/standard/threading/cancellation-in-managed-threads

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/lang/Thread.html

[42] Synchronization

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/lock

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-17.html

[43] Java exception contracts

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-11.html

[44] C# exceptions and filters

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements

[45] C# disposal

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/using

[46] Stack allocation

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/stackalloc

[47] Restricted lifetimes

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/ref-struct

[48] Foreign memory and functions

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/lang/foreign/package-summary.html

[49] Platform invoke

https://learn.microsoft.com/en-us/dotnet/standard/native-interop/pinvoke

[50] Named and optional arguments

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/named-and-optional-arguments

[51] Strings

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/tokens/interpolated

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/tokens/raw-string

https://docs.oracle.com/en/java/javase/27/language/text-blocks.html

[52] Collection construction

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/collection-expressions

https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/List.html

[53] Constructors

https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/tutorials/primary-constructors

https://docs.oracle.com/en/java/javase/27/language/flexible-constructor-bodies.html

[54] Small programs

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/program-structure/top-level-statements

https://docs.oracle.com/en/java/javase/27/language/compact-source-files-instance-main-methods.html

[55] Modules

https://docs.oracle.com/en/java/javase/27/docs/specs/jls/jls-7.html

[56] Assembly access

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/internal

https://learn.microsoft.com/en-us/dotnet/standard/assembly/friend

[57] Attributes and reflection

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/reflection-and-attributes/

[58] Dynamic binding

https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/interop/using-type-dynamic

[59] Source composition and generation

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/partial-classes-and-methods

https://docs.oracle.com/en/java/javase/27/docs/api/java.compiler/javax/annotation/processing/package-summary.html

[60] Package registry scope

https://www.nuget.org/

[61] Maven Central registry

https://central.sonatype.com/

[62] NET publishing

https://learn.microsoft.com/en-us/dotnet/core/deploying/

[63] NET Native AOT

https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/

[64] Java packaging

https://docs.oracle.com/en/java/javase/27/docs/specs/man/jpackage.html

[65] GraalVM Native Image

https://www.graalvm.org/latest/reference-manual/native-image/

[66] Temurin platform coverage

https://adoptium.net/supported-platforms/

https://adoptium.net/about/

[67] Oracle Java support roadmap

https://www.oracle.com/java/technologies/java-se-support-roadmap.html

[68] NET support policy

https://dotnet.microsoft.com/en-us/platform/support/policy

[69] Quarkus packaging and deployment

https://quarkus.io/guides/

[70] ASP NET Core

https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0

[71] Entity Framework Core

https://learn.microsoft.com/en-us/ef/core/

[72] Blazor

https://dotnet.microsoft.com/en-us/apps/aspnet/web-apps/blazor

[73] NET MAUI targets

https://learn.microsoft.com/en-us/dotnet/maui/supported-platforms?view=net-maui-10.0

[74] Java application framework inventory

https://spring.io/projects/

https://quarkus.io/guides/

[75] Data and integration framework inventory

https://spark.apache.org/docs/latest/

https://flink.apache.org/

https://camel.apache.org/

[76] NET game and UI inventory

https://dotnet.microsoft.com/en-us/apps/games

https://avaloniaui.net/

[77] NET core licensing

https://github.com/dotnet/runtime/blob/main/LICENSE.TXT

[78] OpenJDK distribution licensing

https://github.com/openjdk/jdk/blob/master/LICENSE

https://github.com/openjdk/jdk/blob/master/ADDITIONAL_LICENSE_INFO

[79] Oracle JDK licensing policy

https://www.oracle.com/java/technologies/javase/jdk-faqs.html

[80] ASP NET Core licensing

https://github.com/dotnet/aspnetcore/blob/main/LICENSE.txt

[81] Spring Boot licensing

https://github.com/spring-projects/spring-boot/blob/main/LICENSE.txt

[82] Historical C Sharp and Java measurements

https://benchmarksgame-team.pages.debian.net/benchmarksgame/fastest/csharpcore-javavm.html

[83] Benchmark measurement method

https://benchmarksgame-team.pages.debian.net/benchmarksgame/how-programs-are-measured.html

[84] TechEmpower results index

https://www.techempower.com/benchmarks/

[85] TechEmpower test specifications

https://github.com/TechEmpower/FrameworkBenchmarks/wiki/Project-Information-Framework-Tests-Overview